

1 Introdução

A Busca em Largura, ou *Breadth-First Search* (BFS), é um dos algoritmos fundamentais de travessia em grafos [5]. O algoritmo parte de um vértice inicial e, em seguida, visita os demais vértices em camadas: primeiro os que estão a uma aresta de distância, depois os que estão a duas arestas, e assim sucessivamente. Dessa forma, em um grafo não ponderado, a BFS calcula a menor distância, em número de arestas, entre o vértice inicial e todos os vértices alcançáveis a partir dele. A Busca em Largura também gera uma árvore chamada *Árvore de Busca em Largura*, na qual o pai de um vértice é aquele que o descobriu durante a busca.

A implementação clássica utiliza uma fila. O vértice inicial é inserido na fila, marcado como visitado, e sua distância é definida como zero. Em seguida, enquanto a fila não estiver vazia, repete-se a seguinte operação: retira-se um vértice v da fila e examinam-se todos os vértices u vizinhos de v . Caso u ainda não tenha sido visitado, sua distância é calculada, ele é marcado como visitado e adicionado à fila.

Uma forma simplificada do algoritmo é apresentada a seguir:

```
BFS(G, origem):
    para cada vertice v em G:
        pai[v] = none

    pai[origem] = origem
    fila = {origem}

    enquanto fila nao esta vazia:
        u = proximo elemento da fila
        para cada vizinho v de u:
            se pai[v] == none:
                pai[v] = u
                adiciona v na fila
```

2 Problemas Reais

Grafos aparecem naturalmente em problemas reais de grande escala. Em uma rede social, vertices podem representar usuarios e arestas podem representar amizades, seguidores, mensagens ou interacoes [6]. Na Web, paginas sao vertices e hiperlinks sao arestas [4]. Em seguranca computacional, grafos podem modelar fluxos de rede, dependencias entre processos, tentativas de acesso e relacoes entre entidades suspeitas [1]. Em biologia, vertices podem representar proteínas, genes ou moléculas, e arestas podem representar interacoes [2]. Em todos esses casos, a BFS aparece como uma primitiva para calcular alcançabilidade, distancia em numero de arestas e expansao de vizinhanca.

A literatura de sistemas de grafos trata a BFS como uma das operacoes centrais para avaliar plataformas de processamento de dados irregulares. O Graph500 foi proposto para complementar benchmarks numericos tradicionais com uma carga de trabalho orientada a dados, motivado pela diferenca entre

simulacoes fisicas, que tendem a ser estruturadas e intensivas em ponto flutuante, e aplicacoes de informatica, que tendem a ser nao estruturadas, orientadas a inteiros e com baixa localidade [10]. Sua especificacao define a BFS como o Kernel 2 do benchmark, executado sobre grafos sinteticos de escala crescente, e usa a metrica TEPS, ou arestas atravessadas por segundo, para medir desempenho [7]. Isso mostra que a BFS nao e apenas um exercicio teorico: ela e usada como proxy para medir a capacidade de supercomputadores processarem analise de grafos massivos.

Em redes sociais, a BFS modela consultas de vizinhanca e distancia social. A pergunta “quais usuarios estao a ate dois ou tres saltos de uma conta?” e uma execucao limitada de BFS. Esse tipo de consulta aparece em recomendacao de amizades, propagacao de informacao, deteccao de comunidades locais e analise de influencia [6]. Em grafos com bilhoes de relacoes, a expansao de poucos niveis pode atingir uma quantidade enorme de vertices, especialmente quando existem hubs de grau muito alto. Por isso, mesmo uma BFS limitada por profundidade pode exigir processamento paralelo e estruturas de dados distribuidas.

Em grafos da Web, a BFS tambem e uma abstracao natural. Um rastreador *web crawler* pode iniciar em um conjunto de paginas sementes e expandir links por camadas, descobrindo novas paginas a cada nivel [4]. O sistema Pregel, do Google, foi proposto para processamento de grafos em larga escala e apresenta algoritmos como caminhos minimos, PageRank, propagacao de rotulos e outras computacoes iterativas sobre grafos distribuidos [8]. Embora Pregel seja um modelo mais geral que BFS, sua execucao em superpassos e troca de mensagens entre vertices e muito proxima da ideia de fronteiras sucessivas usada em BFS distribuida.

Em seguranca, investigacao de fraudes e analise de dependencias, BFS e usada para expandir a vizinhanca de uma entidade de interesse. A partir de uma conta, um endereco IP, uma maquina infectada ou um processo suspeito, a busca em largura identifica entidades conectadas por poucos saltos. Essa forma de exploracao e importante porque muitas relacoes relevantes nao aparecem em uma aresta direta, mas em cadeias curtas: contas que compartilham dispositivos, maquinas que se comunicam com o mesmo servidor, ou processos que dependem dos mesmos arquivos [1]. Em grafos corporativos ou de telecomunicacoes, essas buscas podem envolver centenas de milhoes de vertices e precisam responder em tempo interativo ou quase interativo.

O desafio e que os grafos modernos podem ser massivos e irregulares. Grafos de redes sociais e da Web frequentemente possuem distribuicao de grau altamente desbalanceada: poucos vertices concentram muitas arestas, enquanto a maioria tem grau baixo. Essa propriedade gera duas dificuldades para BFS. Primeiro, ha desbalanceamento de trabalho, pois expandir um vertice de alto grau custa muito mais do que expandir um vertice comum. Segundo, ha muita contencao, pois muitos caminhos podem tentar descobrir o mesmo vertice ao mesmo tempo.

Alem disso, a BFS tem baixa localidade de memoria. Em matrizes densas ou vetores, os acessos costumam ser contiguos e previsiveis. Em grafos, ao expandir a fronteira, o algoritmo segue listas de adjacencia que podem apontar

para posicoes distantes e aparentemente aleatorias. Consequentemente, a BFS e normalmente limitada por largura de banda, latencia de memoria, atomicos, sincronizacao e comunicacao entre maquinas, e nao por operacoes aritmeticas.

Trabalhos em arquiteturas paralelas mostram como essas dificuldades aparecem na pratica. Beamer, Asanovic e Patterson propuseram a BFS *direction optimizing*, que alterna entre uma expansao top-down, baseada na fronteira atual, e uma expansao bottom-up, que examina vertices ainda nao visitados procurando vizinhos na fronteira [3]. Essa mudanca reduz trabalho em grafos de pequeno diametro e grande fronteira, um caso comum em grafos sociais e da Web. Merrill, Garland e Grimshaw estudaram BFS em GPU e mostraram que a primitiva e representativa de computacoes paralelas com acesso a memoria e distribuicao de trabalho irregulares; sua implementacao alcançou bilhoes de arestas atravessadas por segundo em uma ou mais GPUs [9].

Portanto, BFS aparece em problemas reais de duas formas complementares. Como algoritmo final, ela responde consultas de alcance, distancia e vizinhanca. Como primitiva de sistema, ela estressa exatamente os componentes que tornam grafos massivos dificeis: memoria irregular, fronteiras dinamicas, atomicos, desbalanceamento e comunicacao. Por isso, estudar BFS em GPU e em ambientes distribuidos e uma forma direta de estudar o desempenho de analise de grafos em escala real.

3 BFS em Fronteiras

O algoritmo de BFS também pode ser implementado como uma descoberta por fronteiras. Inicializa-se a primeira fronteira com o vértice inicial. Em seguida, para cada vértice presente na fronteira atual, examinam-se todos os seus vizinhos; aqueles que ainda não foram visitados são adicionados à próxima fronteira. Esse processo se repete sucessivamente, até que a fronteira atual esteja vazia.

Uma versão simplificada do algoritmo é apresentada a seguir:

```
BFS(G, origem):
    para cada vertice v em G:
        pai[v] = none

    pai[origem] = origem
    fronteira = {origem}

    enquanto fronteira nao esta vazia:
        proxima_fronteira = vazia
        para cada vertice u na fronteira:
            para cada vizinho v de u:
                se pai[v] == none:
                    pai[v] = u
                    adiciona v na proxima_fronteira
        fronteira = proxima_fronteira
```

3.1 Exemplo Visual de Fronteiras

Considere o grafo da Figura 1. A busca começa no vertice A. Em cada passo, os vertices em azul representam a fronteira atual, os vertices em cinza já foram visitados em níveis anteriores, e os vertices em branco ainda não foram descobertos. As arestas destacadas em azul formam a árvore de BFS construída até aquele momento. Essa árvore registra, para cada vertice descoberto, qual foi o vertice responsável por sua primeira descoberta. Assim, cada aresta azul liga um vertice ao seu pai na busca, permitindo recuperar um caminho mínimo, em número de arestas, da origem A até qualquer vertice alcançável.

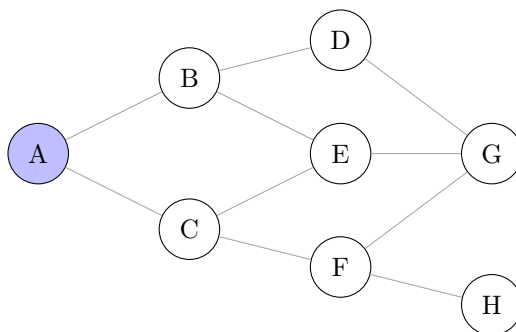


Figure 1: Passo 0 da BFS: a fronteira inicial contém apenas a origem A.

No passo inicial, a distância de A é zero. A primeira expansão examina as arestas que saem de A. Como B e C ainda não foram visitados, ambos são descobertos, recebem distância um e passam a formar a próxima fronteira.

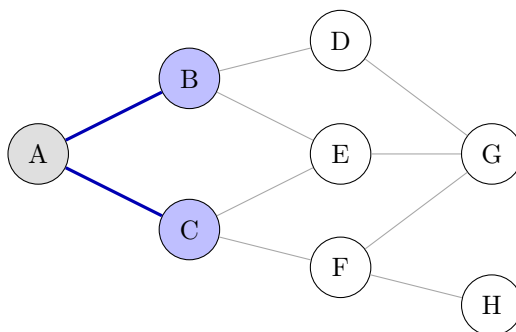


Figure 2: Passo 1 da BFS: B e C formam a fronteira de nível 1.

No segundo passo, a BFS expande todos os vértices da fronteira atual. O vértice B descobre D e E. O vértice C também possui uma aresta para E, mas E já pode ter sido descoberto por B; nesse caso, a implementação mantém apenas um pai para E. O vértice C também descobre F. Assim, a nova fronteira passa a ser formada por D, E e F, todos com distância dois.

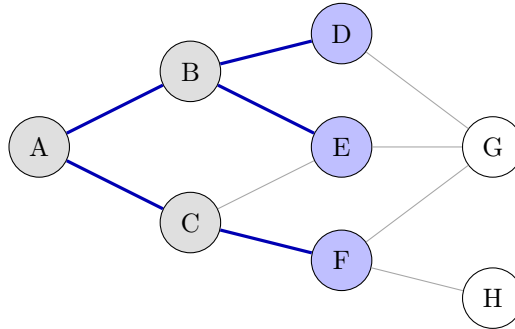


Figure 3: Passo 2 da BFS: D, E e F formam a fronteira de nível 2.

No terceiro passo, a fronteira de nível dois é expandida. Os vértices D, E e F possuem arestas para G. O primeiro que conseguir registrar G como descoberto define seu pai na árvore de BFS. Na figura, E foi escolhido como pai de G, mas D ou F também poderiam ser pais válidos, pois todos pertencem ao nível anterior. Além disso, F descobre H, que também passa a fazer parte da fronteira de nível três.

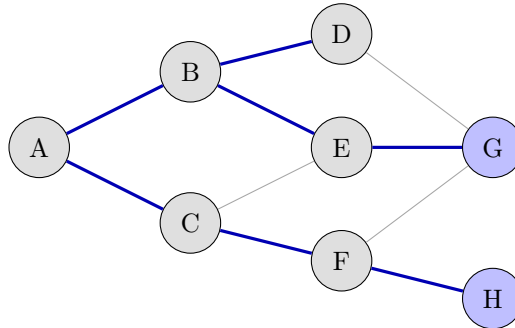


Figure 4: Passo 3 da BFS: G e H são descobertos no nível 3.

Depois da expansão de G, nenhum novo vértice é encontrado. A próxima fronteira fica vazia e o algoritmo termina. O resultado é uma árvore de BFS na qual cada vértice alcançável possui um pai em um nível imediatamente anterior ao seu.

4 Motivação de Paralelismo

A BFS baseada em fronteiras oferece uma oportunidade natural de paralelismo, todos os vértices de uma mesma fronteira podem ser processados simultaneamente, e suas arestas podem ser expandidas em paralelo. O principal cuidado é controlar a descoberta dos vértices da próxima fronteira, pois múltiplas arestas

podem tentar descobrir o mesmo vertice ao mesmo tempo. Estrategias de paralelismo e controle foram discutidas em [9].

4.1 Divisão do Grafo entre Workers

Neste texto, o termo *worker* é usado de forma geral para representar uma unidade de execução capaz de armazenar parte do grafo e realizar parte do trabalho da BFS. Um worker pode corresponder, por exemplo, a um processo, uma máquina, uma GPU, uma CPU ou outro recurso de execução paralelo. Os detalhes de como esses workers são implementados serão discutidos posteriormente. Essa abstração é comum em sistemas de processamento paralelo de grafos, nos quais a computação é dividida entre unidades que armazenam partes do grafo e trocam informações sobre vértices descobertos durante a execução [9].

Em uma implementação paralela ou distribuída, o grafo não precisa ficar inteiro em uma única unidade de execução. Uma estratégia comum é dividir seus vértices entre vários *workers*. Cada worker fica responsável por armazenar e processar uma parte dos vértices e de suas listas de adjacência. Durante a BFS, quando um worker expande um vértice local, ele examina suas arestas; se o vizinho pertence a outro worker, a descoberta desse vizinho precisa ser comunicada ao worker que o possui [9].

Uma forma simples de particionamento é a distribuição cíclica. Com n_w workers, o dono de um vértice pode ser calculado por:

$$\text{worker}(v) = v \bmod n_workers$$

Assim, os vértices são distribuídos de maneira alternada entre os workers. Essa estratégia não garante balanceamento perfeito para todos os grafos, mas é simples e evita concentrar intervalos inteiros de vértices em um único worker. A Tabela 1 mostra um exemplo com 16 vértices.

Worker	Vértices pertencentes ao worker
0	0, 4, 8, 12
1	1, 5, 9, 13
2	2, 6, 10, 14
3	3, 7, 11, 15

Table 1: Particionamento cíclico de 16 vértices entre 4 workers.

A Figura 5 mostra a visão global de um grafo com 16 vértices. Essa visão representa o grafo completo antes de considerar qual worker é responsável por cada vértice.

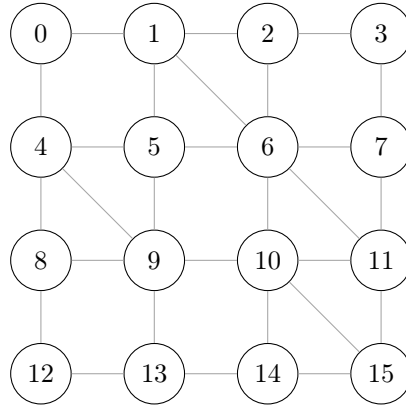


Figure 5: Visão global de um grafo com 16 vértices antes do particionamento.

A Figura 6 mostra o mesmo grafo particionado entre quatro workers. As cores indicam o dono de cada vértice. As arestas que ligam vértices de cores diferentes representam comunicações potenciais durante a BFS, pois a expansão de um vértice em um worker pode descobrir um vértice que pertence a outro worker.

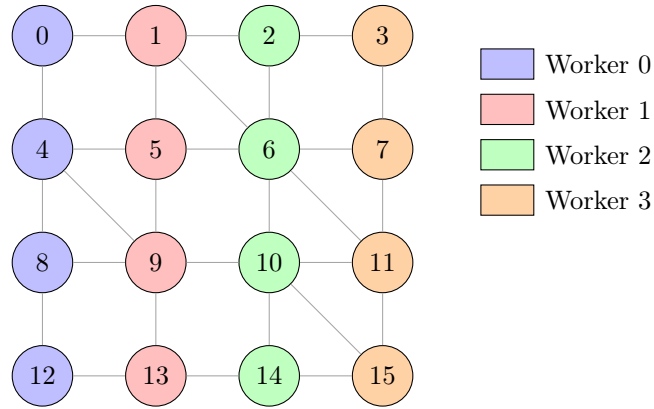


Figure 6: Mesmo grafo particionado entre 4 workers usando a regra $worker(v) = v \bmod 4$.

4.2 Exemplo de BFS Paralela com 4 Workers

Considere uma BFS iniciada no vértice 0 do grafo particionado da Figura 6. No passo inicial, apenas o worker 0 possui trabalho, pois ele é o dono do vértice 0. Ao expandir 0, são descobertos os vértices 1 e 4. O vértice 4 pertence ao próprio worker 0, enquanto o vértice 1 pertence ao worker 1. Portanto, a nova fronteira já fica distribuída entre mais de um worker.

Após a expansão dos vértices 1 e 4, a BFS chega ao passo mostrado na Figura 9. Nesse momento, a fronteira global de nível 2 é formada por vértices armazenados em workers diferentes. O worker 0 possui o vértice 8 na fronteira, o worker 1 possui os vértices 5 e 9, o worker 2 possui os vértices 2 e 6, e o worker 3 ainda não possui vértices na fronteira atual.

A expansão passo a passo é mostrada nas figuras seguintes. Em cada uma delas, a fronteira global é dividida entre os quatro workers. Os vértices em azul formam a fronteira local do passo atual, os vértices em cinza já foram visitados, e os vértices em branco ainda não foram descobertos. Depois que todos os workers expandem suas fronteiras locais, os vértices recém-descobertos formam a fronteira do próximo nível.

No passo 0, apenas o worker 0 possui trabalho, pois a origem 0 pertence a ele.

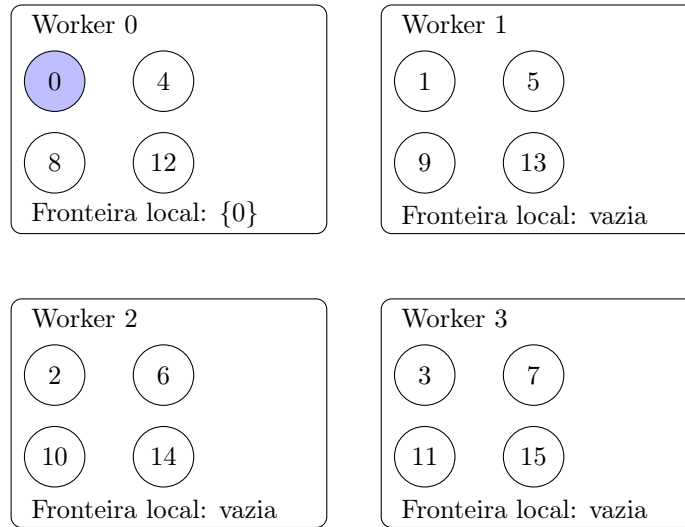


Figure 7: Passo 0 da BFS paralela: apenas a origem 0 está na fronteira.

Ao expandir 0, são descobertos 1 e 4. O vértice 4 permanece no worker 0, e o vértice 1 passa a compor a fronteira local do worker 1.

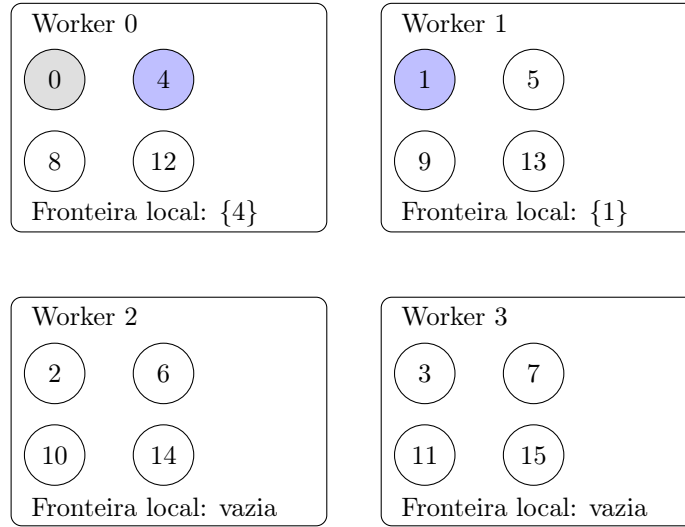


Figure 8: Passo 1 da BFS paralela: os workers 0 e 1 possuem vértices na fronteira.

No passo 1, os workers 0 e 1 trabalham ao mesmo tempo: 4 descobre 8 e 9, enquanto 1 descobre 2, 5 e 6. O vértice 5 também é vizinho de 4, mas basta que uma das tentativas de descoberta seja aceita para que ele entre na próxima fronteira. No passo 2, já há trabalho em três workers diferentes.

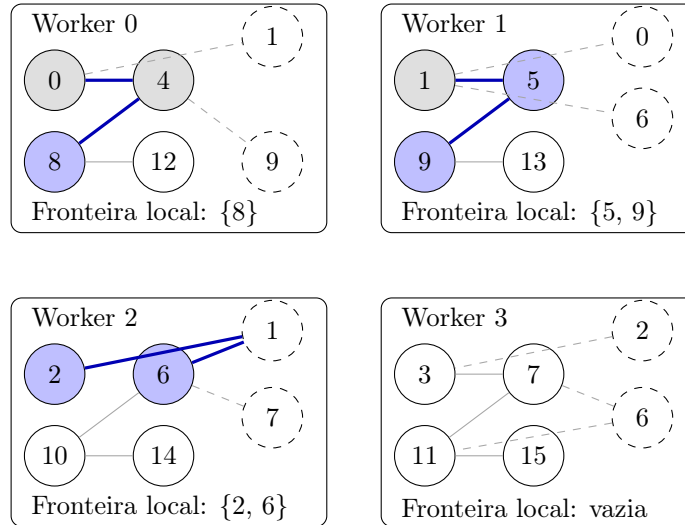


Figure 9: Passo 2 da BFS paralela: a fronteira global está distribuída entre os workers 0, 1 e 2. Vértices tracejados representam vizinhos remotos mantidos apenas para ilustrar comunicações entre workers.

No passo 2, a expansão dos vértices 2 e 6 no worker 2 gera trabalho para o worker 3, pois existem arestas para os vértices 3, 7 e 11. Ao mesmo tempo, os workers 0, 1 e 2 descobrem outros vértices locais, formando a próxima fronteira.

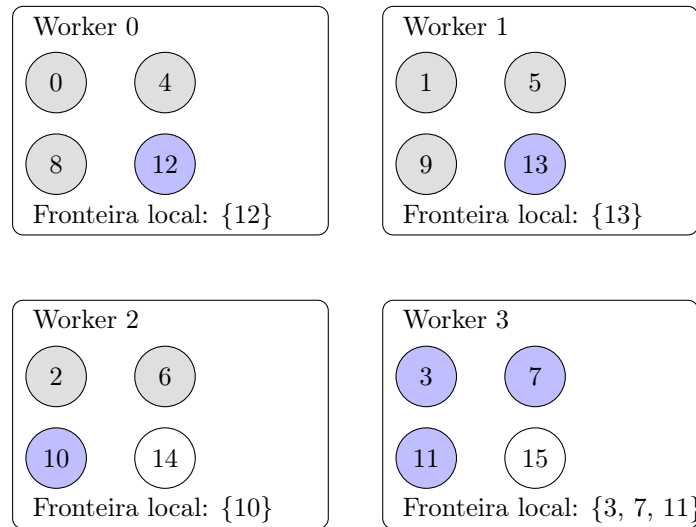


Figure 10: Passo 3 da BFS paralela: todos os workers possuem vértices na fronteira.

Após o passo 3, apenas os workers 2 e 3 continuam com novos vértices a expandir.

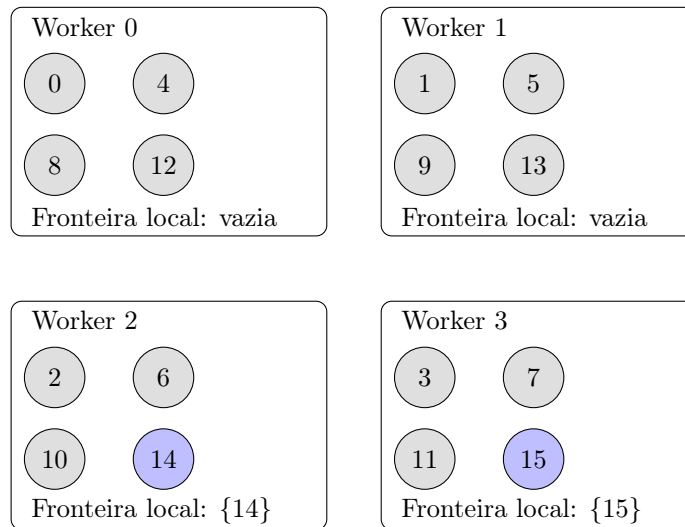


Figure 11: Passo 4 da BFS paralela: apenas os workers 2 e 3 continuam com fronteiras locais.

Por fim, a expansão de 14 e 15 não produz novas descobertas. Nesse ponto, a próxima fronteira fica vazia e a BFS termina.

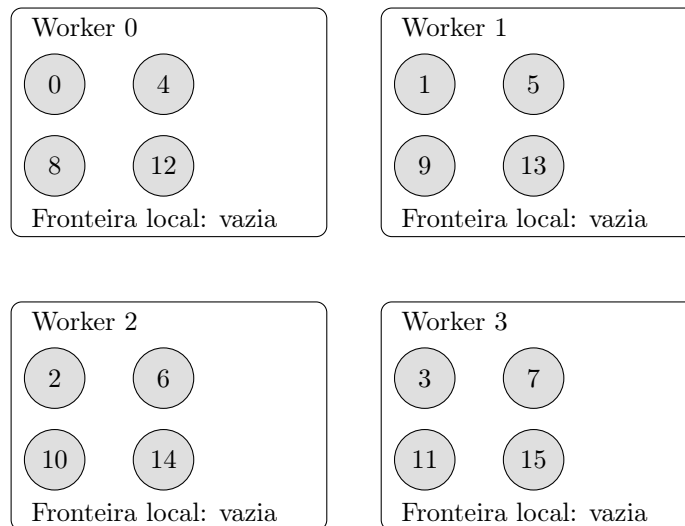


Figure 12: Passo 5 da BFS paralela: a fronteira global fica vazia e a busca termina.

Essa sequência visual mostra que a fronteira global da BFS é a união das fronteiras locais de cada worker. Em uma implementação paralela, cada worker ex-

pande apenas os vértices da sua fronteira local. Quando uma aresta aponta para um vértice remoto, a atualização precisa ser enviada ao worker dono daquele vértice, que poderá processá-lo no próximo passo. Essa organização preserva a ideia de avanço por níveis da BFS, mas distribui o trabalho e a comunicação entre as unidades de execução disponíveis.

5 BFS em GPU

GPUs modernas são otimizadas para executar operações computacionalmente intensivas com muitos grupos de threads em paralelo e dependem de concorrência para esconder atrasos de memória. No modelo da NVIDIA, threads são agrupadas em *warps*, que executam uma mesma instrução sobre dados diferentes, e em blocos de threads, que podem compartilhar memória local no multiprocessador. Esse modelo favorece cargas de trabalho com grande quantidade de operações independentes e com acesso a memória bem organizado [9].

A BFS é atraente para GPUs porque a expansão de uma fronteira pode expor muito paralelismo. Em uma mesma iteração, vários vértices da fronteira atual podem ser expandidos simultaneamente, e as arestas incidentes a esses vértices também podem ser examinadas em paralelo. Quando a fronteira é grande, há trabalho suficiente para ocupar muitos multiprocessadores. Além disso, a BFS executa pouca computação aritmética por aresta; portanto, seu desempenho depende principalmente de largura de banda, latência de memória, controle de concorrência e eficiência na organização das fronteiras [9].

Apesar disso, grafos esparsos tornam a BFS difícil em GPU. O padrão de acesso a memória depende da estrutura do grafo, não de um arranjo regular como em vetores ou matrizes densas. Ao expandir um vértice, as threads seguem listas de adjacência que podem apontar para regiões distantes da memória. Além disso, grafos reais frequentemente possuem distribuição de grau irregular, alguns vértices têm poucos vizinhos, enquanto outros concentram muitos. Se uma thread ou um warp recebe um vértice de grau muito maior que os demais, ocorre desbalanceamento de carga e parte da GPU pode ficar subutilizada [9].

Uma forma simples de paralelizar BFS é fazer cada iteração inspecionar todos os vértices ou todas as arestas do grafo, procurando quais deles pertencem ao nível atual. Essa estratégia se ajusta facilmente ao modelo de dados paralelos da GPU, mas pode executar trabalho quadrático no pior caso, pois o mesmo grafo é varrido repetidas vezes ao longo dos níveis da busca. Para grafos com diâmetro não trivial, esse custo se torna alto. Uma implementação eficiente deve examinar apenas os vértices e arestas que pertencem às fronteiras ativas de cada iteração, mantendo custo linear em relação ao número de vértices e arestas, isto é, $O(|V| + |E|)$ [9].

No ponto de vista de desempenho, permitir muitas descobertas duplicadas de um mesmo vértice pode gerar trabalho redundante, pois o mesmo vértice pode ser inserido mais de uma vez na fronteira e sua lista de adjacência pode ser expandida repetidamente. Por isso, implementações em GPU costumam usar mecanismos de filtragem, marcação e remoção local de duplicatas para reduzir

esse efeito [9].

Portanto, a BFS em GPU deve equilibrar tres objetivos. Primeiro, preservar a estrutura por niveis da BFS, garantindo que a proxima fronteira contenha apenas vertices descobertos a partir da fronteira atual. Segundo, manter eficiencia de trabalho, evitando varrer o grafo inteiro a cada iteracao. Terceiro, adaptar a execucao irregular do grafo ao modelo de hardware da GPU, organizando fronteiras, trabalho e comunicacao de forma a reduzir desbalanceamento, contencao e acessos inefficientes a memoria [9].

References

- [1] Leman Akoglu, Hanghang Tong, and Danai Koutra. Graph based anomaly detection and description: A survey. *Data Mining and Knowledge Discovery*, 29(3):626–688, 2015.
- [2] Albert-Laszlo Barabasi and Zoltan N. Oltvai. Network biology: Understanding the cell’s functional organization. *Nature Reviews Genetics*, 5(2):101–113, 2004.
- [3] Scott Beamer, Krste Asanovic, and David Patterson. Direction-optimizing breadth-first search. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE Computer Society Press, 2012.
- [4] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. In *Proceedings of the Seventh International Conference on World Wide Web*, pages 107–117. Elsevier Science Publishers B. V., 1998.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [6] David Easley and Jon Kleinberg. *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press, 2010.
- [7] Graph 500 Steering Committee. Graph 500 Benchmark Specification: Graph 500 Benchmarks 1 (Search) and 2 (Shortest Path), 2017. Acesso em: 25 jun. 2026.
- [8] Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: A system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, pages 135–146. ACM, 2010.
- [9] Duane Merrill, Michael Garland, and Andrew Grimshaw. Scalable gpu graph traversal. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 117–128. ACM, 2012.

- [10] Richard C. Murphy, Kyle B. Wheeler, Brian W. Barrett, and James A. Ang. Introducing the graph 500. In *Cray User's Group*. Sandia National Laboratories, may 2010. SAND2010-3263C. Acesso em: 25 jun. 2026.